

5V supply to this camera.

SPI communication. SPI communication with MRF24WB0MA is illustrated in Fig. 6. PIC microcontroller is chosen as master and Wi-Fi as slave. FOSC/64 clock mode is chosen by proper setting of SSPCON1 register of PIC18F4620.

The software

Software for the PIC microcontroller is written in 'C' language and compiled using MPLAB IDE. The program flowchart is shown in Fig. 7.

The project is developed using 'C' source files such as WF_Spi, ARP, DHCP, ICMP, IP, main, stackTSK, TCPIP, tick, UART, UDP, WF_Config, WF_Eint, WFConnectionAlgorithm, WFConnectionManager, WFConnectionProfile, WFDataTxRx, WFDriverCom, WFDriverRaw, WFEventHandler, WFInit, WFMac, WFMgmtMsg, WFParamMsg, WFPowerSave, WFS-can, WFTxPower and xlcd. (All these files are included in this month's EFY EVD.)

The project is based on communication between the Wi-Fi module, microcontroller, PC and the camera, which are closely controlled through the software. When LED2 starts blinking it indicates that stacking is running and communication is successfully established with remote client PC. This LED2 monitored through timer interrupt is defined in main.c file.

The Wi-Fi driver routines are nothing but routine files from Microchip TCP-IP stack. The ARP, ICMP, UDP and DHCP protocol implementation are all files from Microchip TCP-IP stack. But the TCP protocols are customised routines written in 'C' to incorporate camera routines into the memory size of PIC18F4620. The TCP protocol implementation and camera routines are available in IP.c source file only.

Before writing the code note that 10MHz crystal is used as oscillator source and the clock is set at 40 MHz using HSPLL fuse setting through software.

```
#pragma config OSC=HSPLL, WDT=OFF,
MCLR=ON, PBAEN=OFF, LVP=OFF
```

Below some important source program routines used in this project are explained.

SPI communication. SPI communication between PIC and Wi-Fi is initialised by suitable initiation code before starting communication. These instructions are available in WF_Spi.c file as given below.

```
WF_SPICON1 = 0x30; // SSPEN bit is
set, SPI in master mode, (0x30 is for
POSC/4),
```

```
WF_SPISTATbits.CKE = 0; // Transmit
data on falling edge of clock
```

```
WF_SPISTATbits.SMP = 1; // Input
sampled at end? of data output time
```

After SPI initialisation, Wi-Fi should be initialised for proper network protocol transmission.

Wi-Fi initialisation and other relevant codes are included in the WFMac.c file. The routines listed below are implemented only after understanding the coding in Microchip TCP-IP Stack.

MACInit () routine. MACInit () routine given below does necessary initialisation of RAW registers in Wi-Fi module.

```
void MACInit(void)
{
    WF_Init();
}
```

MACInit() calls WF_Init() and RawInit() as given below.

The WFWareInit() in WF_Init() is important for proper initialisation.

WF_Init routine is given below:

```
void WF_Init(void)
{
    UINT8 version;
    unsigned char LCD1[16];
    WFWareInit();
    RawInit();
    WFGGetMRF24WB0MVersion(&version);
    WF_ASSERT(version >= EXPECTED_
MRF24WB0M_VERSION_NUMBER);
    WFWEnableMRF24WB0MMode();
    WF_LibInitialize();
}
```

RAWInit() routine is given below:

```
void RawInit(void)
{
```

```
ScratchUnmount(RAW_TX_ID);
ScratchMount(RAW_RX_ID);
ScratchUnmount(RAW_RX_ID);
RawWindowReady(RAW_RX_ID) = FALSE;
g_encPtrRAWID[ENC_RD_PTR_ID] =
RAW_RX_ID;
g_encIndex[ENC_RD_PTR_ID] = BASE_
TCB_ADDR;
RawWindowReady(RAW_TX_ID) = FALSE;
SetRawWindowState(RAW_TX_ID, WF_RAW_
UNMOUNTED);
g_encPtrRAWID[ENC_WT_PTR_ID] = RAW_
INVALID_ID;
g_encIndex[ENC_WT_PTR_ID] = BASE_
TX_ADDR
g_wasDiscarded = TRUE;
g_rxBufferSize = 0;
g_txPacketLength = 0;
g_txBufferFlushed = TRUE;
g_rxIndexSetBeyondBuffer = FALSE;
return dwReturn;
}
```

Camera interface. The camera should be first connected to PIC microcontroller via serial port. The baud rate can be 14400, 56000, 57600 or 115200, though better synchronisation was obtained at baud rate 56000 during testing. The camera is assigned with dynamic IP address, which is assigned by DHCP client routine rather than fixed IP address. The advantage is that the IP address gets assigned automatically as per local network and we need not assign it manually. DHCP client routine associated with UDP protocol does the necessary task for auto configuration of the IP address.

The camera should be synchronised by issuing sync command until it acknowledges c328syn() routine, which is part of the IP.c file, as listed below. After taking pictures, suitable reset command is issued to camera module for taking fresh pictures. The Reset() routine is also included in the IP.c file for the same.

The sync command is sent as given below:

```
void c328syn() {
    BYTE a[6], i;
    char k[6];
    sl:
    do {
        send(C328R_COMMAND_PREFIX);
```